

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



III. EDs Não-Lineares

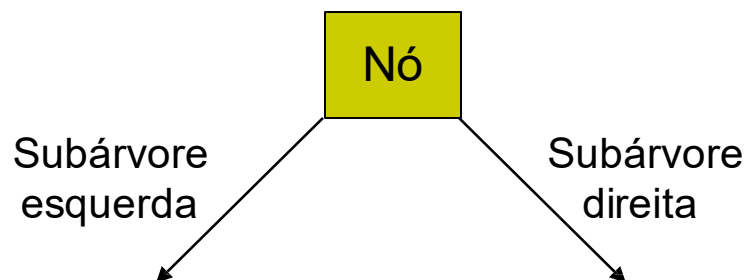


III. EDs Não-Lineares

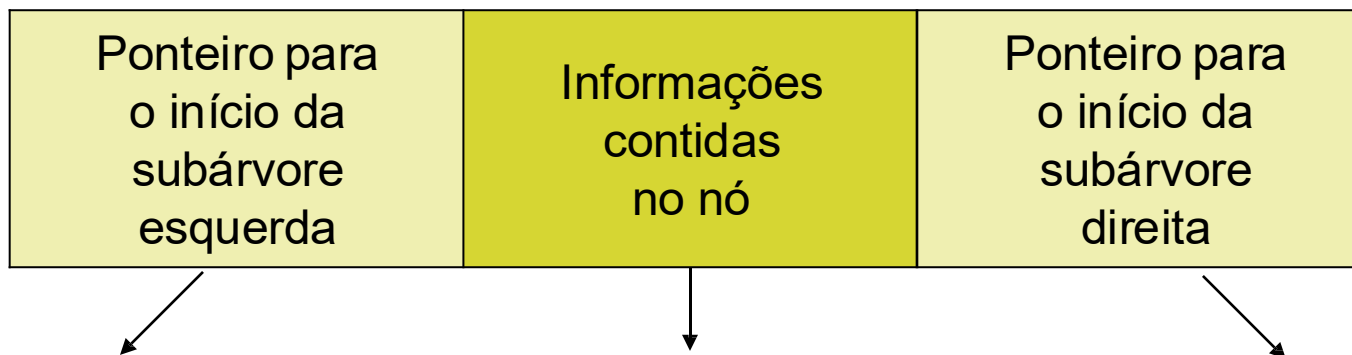
1. Introdução às Estruturas Não-Lineares
2. Conceitos e Terminologia
3. Representações de Árvores
4. Árvores Binárias
5. Percursos em Árvores Binárias
6. Árvores de Busca Binária (BST)
7. Árvores Balanceadas
8. Árvores B e B+
9. Comparações e Aplicações

4. Árvores Binárias

- Todos os nós de uma árvore binária têm no máximo **duas** subárvores **grau 2**



- Representação do nó (alocação dinâmica):



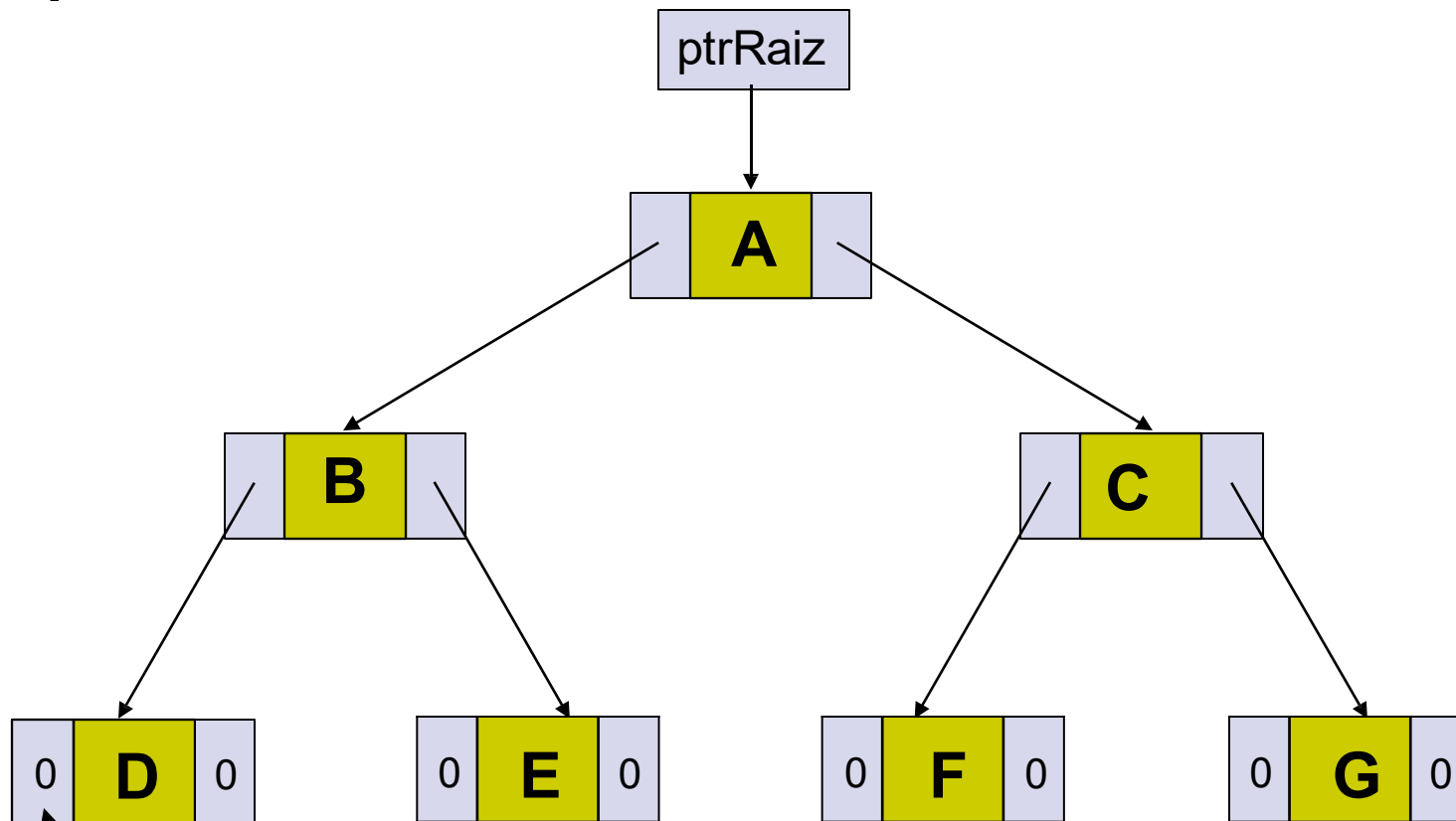


4. Árvores Binárias

```
struct Noh
{
    int dado;
    struct Noh *esquerda;
    struct Noh *direita;
};
```

4. Árvores Binárias

- Exemplo:



Ponteiro nulo (não tem sub-árvore)

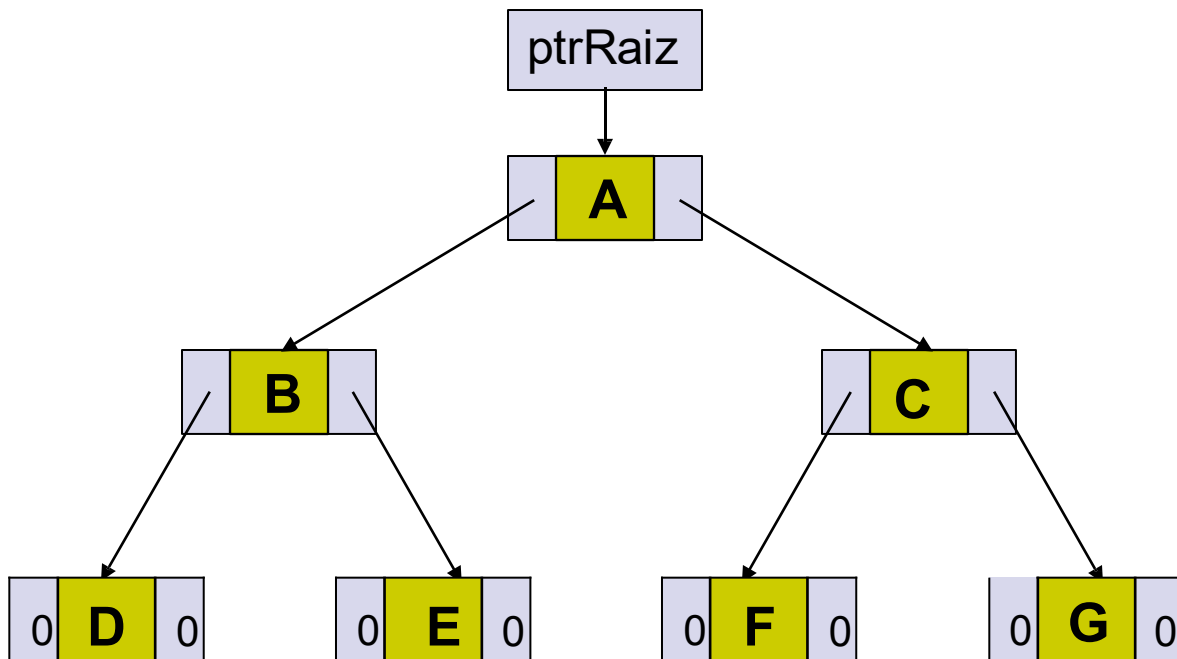


5. Percurso em árvores binárias

- **Percorrer** uma árvore binária significa processar cada nó **apenas uma vez**
- Três ordens de percurso são mais comuns:
 - Pré-ordem
 - Simétrica (ou “em ordem”)
 - Ordem final (ou “pós-ordem”)

5. Percurso em pré-ordem

- Sequência do percurso:
 - Nó raiz → subárvore esquerda em pré-ordem → subárvore direita em pré-ordem



Percurso:

A B D E C F G

Percurso inverso:

A C G F B E D

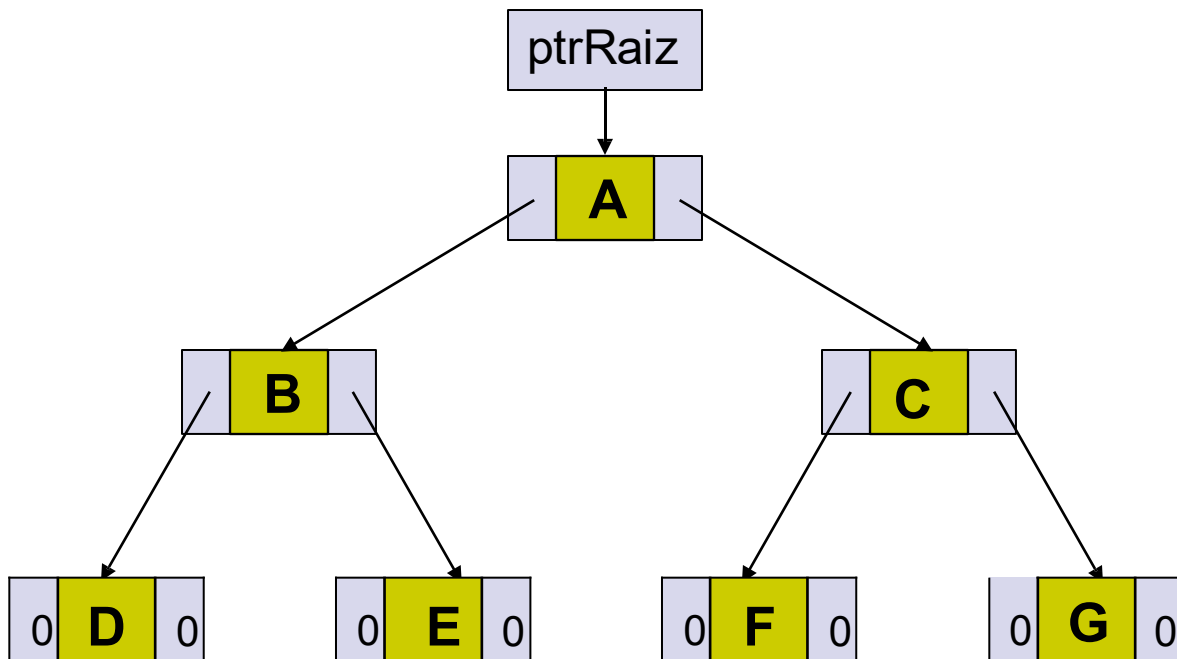


5. Percurso em pré-ordem

```
void exibirPreOrdem(Noh * & pRaiz){  
    if(pRaiz != NULL){  
        printf("\n%d", pRaiz->dado);  
        exibirPreOrdem(pRaiz->esquerda);  
        exibirPreOrdem(pRaiz->direita);  
    }  
}
```

5. Percurso em ordem (simétrica)

- Sequência do percurso:
 - Subárvore esquerda em ordem simétrica → Nó raiz → subárvore direita em ordem simétrica



Percurso:

D B E A F C G

Percurso inverso:

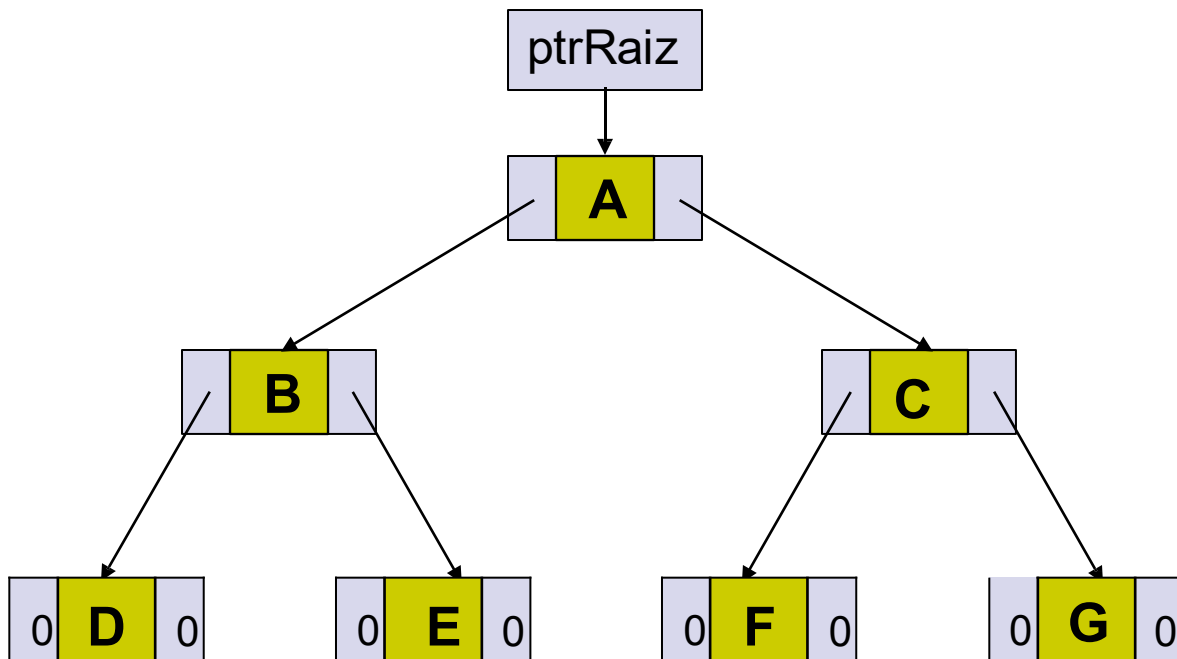
G C F A E B D

5. Percurso em ordem (simétrica)

```
void exibirEmOrdem(Noh * & pRaiz){  
    if(pRaiz != NULL){  
        exibirEmOrdem(pRaiz->esquerda);  
        printf("\n%d", pRaiz->dado);  
        exibirEmOrdem(pRaiz->direita);  
    }  
}
```

5. Percurso em pós-ordem (final)

- Sequência do percurso:
 - Subárvore esquerda em ordem final → subárvore direita em ordem final → Nó raiz



Percurso:

D E B F G C A

Percurso inverso:

G F C E D B A

5. Percurso em pós-ordem (final)

```
void exibirPosOrdem(Noh *pRaiz){  
    if(pRaiz != NULL){  
        exibirPosOrdem(pRaiz->esquerda);  
        exibirPosOrdem(pRaiz->direita);  
        printf("\n%d", pRaiz->dado);  
    }  
}
```

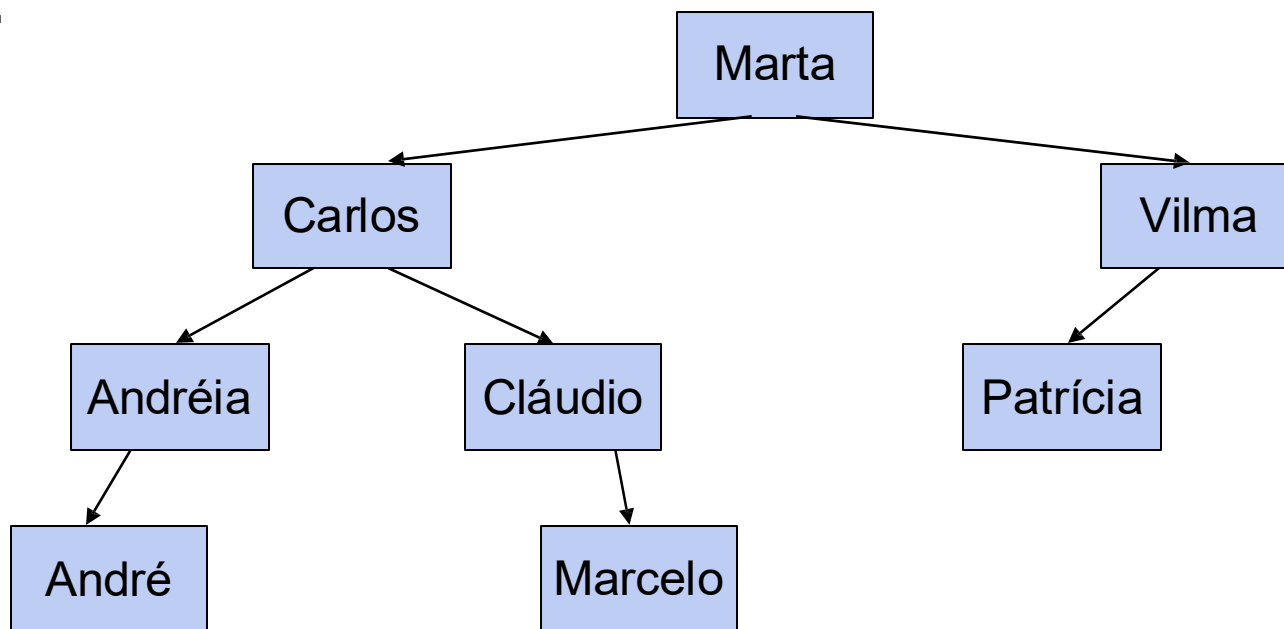


6. Árvore de Busca Binária (ABB)

- Se os elementos da árvore binária **não estão organizados**, a busca consiste num simples **percurso**:
 - até encontrar o elemento desejado (caso exista)
 - até concluir que o elemento não existe (todos os nós inspecionados)
- Conceitualmente, é o mesmo que fazer **busca linear** em um vetor

6. Árvore de Busca Binária (ABB)

- Exemplo:



- O percurso em **ordem simétrica** resultará nos elementos ordenados alfabeticamente

6. Árvore de Busca Binária (ABB)

- Dados:
 - uma certa chave de busca **c**
 - um certo nó atual **A**, com chave **a**
- Uma das situações acontece:
 - **c = a** (a busca termina)
 - **c < a** (a busca é reiniciada com a subárvore à esquerda de **A**, se existir)
 - **c > a** (a busca é reiniciada com a subárvore à direita de **A**, se existir)
 - A busca termina por inexistência da subárvore

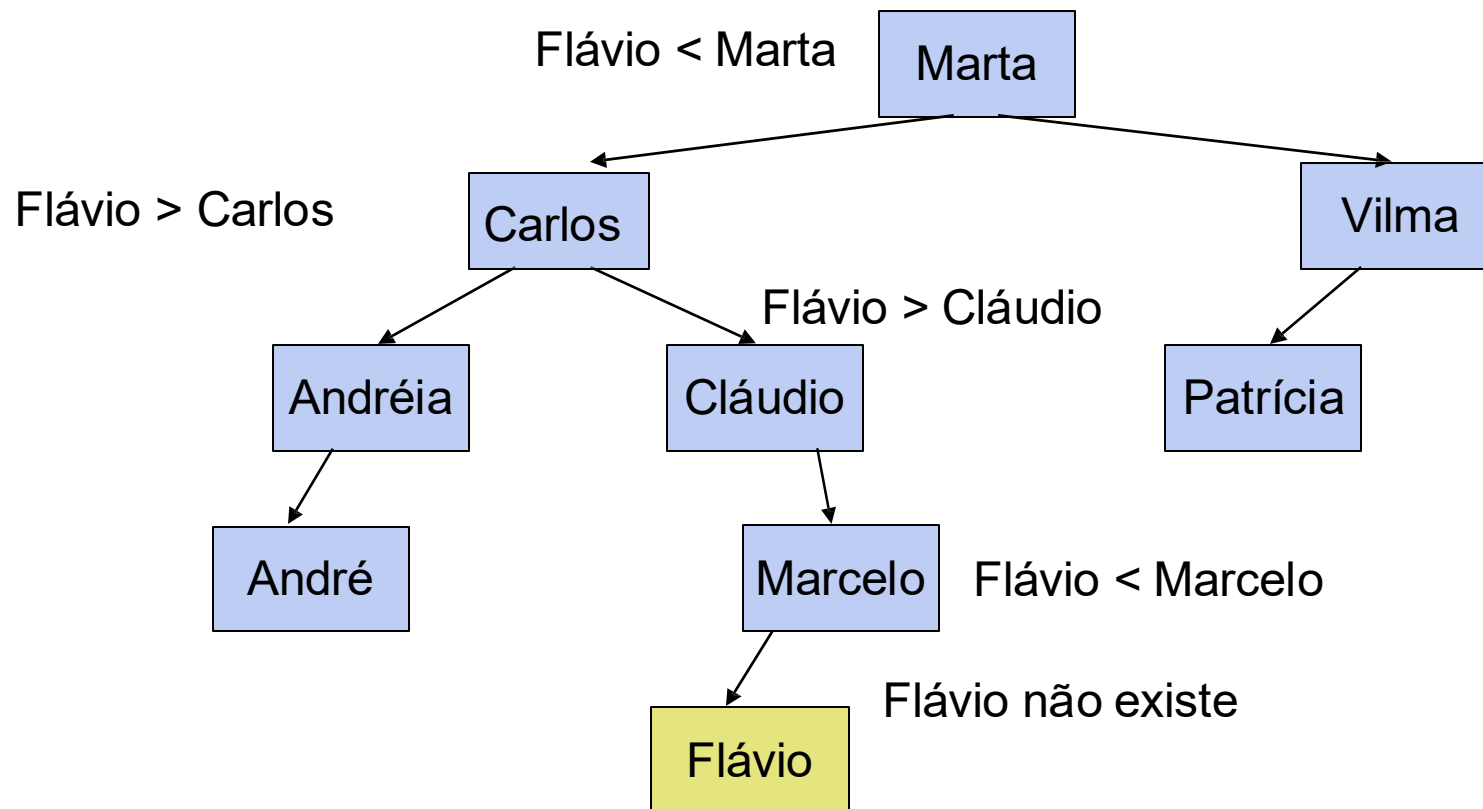


6. Inserção numa ABB

- É assumido que:
 - Não são aceitas chaves duplicadas → busca
 - O nó inserido será sempre uma **folha** da ABB
- Assim, se não existe ainda a chave na ABB:
 - Basta ligar o novo nó ao **último utilizado** na busca, à esquerda ou à direita (dependendo de seu valor de chave)
 - Isto mantém a organização da ABB

6. Inserção numa ABB

- Exemplo: inserir **Flávio**

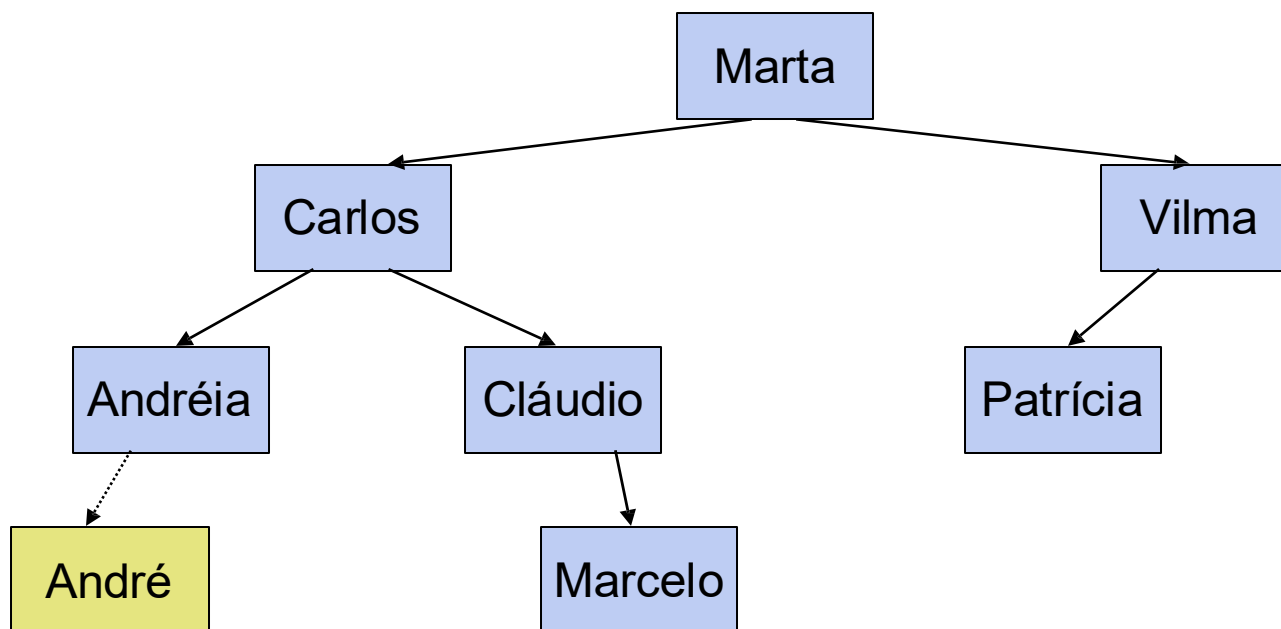


6. Inserção numa ABB

```
void insercao(Noh * & pRaiz, int dado)
{
    if (pRaiz == NULL)
    {
        pRaiz = (Noh *) calloc(1, sizeof (Noh));
        pRaiz->esquerda = NULL;
        pRaiz->direita = NULL;
        pRaiz->dado = dado;
    }else{
        if (dado < (pRaiz->dado)){
            insercao(pRaiz->esquerda, dado);
        }else{
            insercao(pRaiz->direita, dado);
        }
    }
}
```

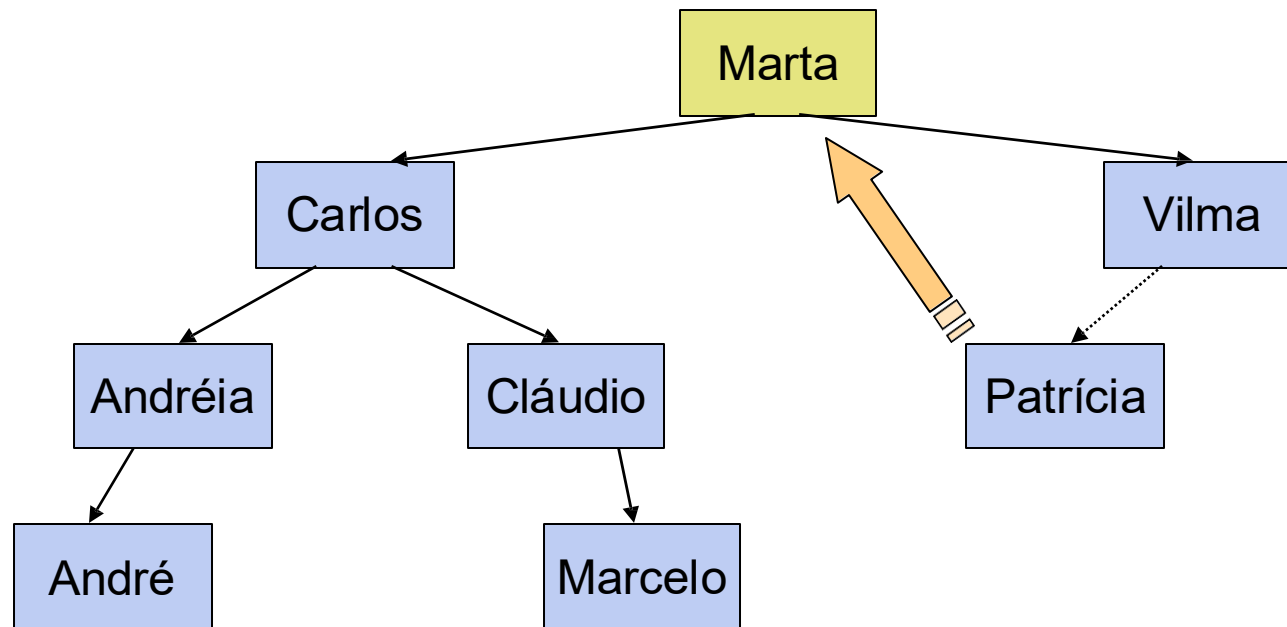
6. Remoção numa ABB

- Um elemento só pode ser retirado se já pertencer à árvore: é preciso fazer busca
- Se elemento é um **nó-folha**:
 - Basta atualizar o ponteiro para subárvore de seu **nó-pai**



6. Remoção numa ABB

- Se elemento **não é** um nó-folha:
 - Mover esse elemento até ele se tornar nó-folha
 - Os nós encontrados no caminho são movidos sem afetar a organização da ABB





Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva